



ANKARA YILDIRIM BEYAZIT UNIVERSITY

GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

COMPUTER ENGINEERING

DataGen: Matrix Dataset Generation

By

Mert Altekin 21050111065

Ali Emre Pamuk 21050111021

Faruk Kaplan 21050111026

Supervised by

Fahreddin Şükrü Torun

Date

12.10.2024

ABSTRACT

The project focuses on developing new matrices of varying sizes with similar properties using various AI based techniques. These matrices, derived from public datasets, will be optimized through a diversified solution strategies to address computational challenges in large-scale simulations and sparse linear systems. Trained on the various matrix resources, the model enhances computation speed and accuracy, reducing time and improving optimization, as validated by empirical experiments.

Table of Contents

1. INTRODUCTION	4
2. METHODOLOGY / SYSTEM DESIGN	5
3. PROJECT MANAGEMENT	7
4. FUTURE WORKS	8
LIST OF FIGURES	9
REFERENCES	10

INTRODUCTION

Background:

This project addresses the computational challenges of large-scale matrix processing by developing a AI based approach to generate matrices of varying sizes while maintaining essential properties. Efficient matrix generation is needed across fields such as data analysis, scientific research, and machine learning.

Problem Statement:

As input matrices grow in size and complexity, traditional generation techniques struggle to scale and optimize for computational efficiency. This project proposes using diversified different techniques to dynamically generate input matrices that retain essential properties and reducing manual tuning and improving computational performance.

- **Related Work:**

RMAT graph generator : D. Chakrabarti, Y. Zhan and C. Faloutsos, R-MAT: A Recursive Model for Graph Mining, pp. 442-446.

- SPARSKIT: a basic tool kit for sparse matrix computations VERSION 2 Youcef Saad*
- Finite Element Matrix Generation On A GPU: A. Dziekonski , P. Sypek, A. Lamecki, and M. Mrozowski
- J-Orthogonal Matrices: Properties and Generation Nicholas J. Higham†

Objectives:

1. **Matrix Generation:** Develop an AI-based method to generate new matrices from existing ones while preserving critical properties and varying in size.
2. **Solver Efficiency:** Apply several techniques to predict optimal solution properties in parallel hybrid solvers for sparse linear systems, improving both accuracy and computation time.
3. **Expected Outcomes:** Achieve faster computations, improved accuracy in matrix generation, and a framework for parallelizing optimization with minimal manual intervention.

Resources for Matrix Generation:

Tools like ANSYS, MATLAB, and SciPy offer robust libraries for generating and manipulating matrices, including sparse systems. The SuiteSparse Matrix Collection provides a large repository of real-world sparse matrices for benchmarking and training models.

1. METHODOLOGY / SYSTEM DESIGN

The system is designed with several key modules to streamline matrix generation, optimization, and solving sparse linear systems. The process leverages AI-based solutions and parallel computation techniques to address computational challenges efficiently.

Matrix Generation Module:

This module collects matrices from publicly available sources, such as the SuiteSparse Matrix Collection (e.g., sparse.tamu.edu), and preprocesses them for use. It will also fetch matrices with similar properties from applications like ANSYS, allowing for a diverse range of matrix types. Matrices will typically be evaluated based on key features such as non-zero elements, row and column statistics, and symmetry. These properties will be considered to optimize the matrix generation process (Fig 1.3). Additionally, image processing techniques will be employed to upscale or downscale matrix patterns, allowing the system to create matrices of varying sizes and complexities while maintaining the essential characteristics of the original patterns. (Fig 1.1)

Optimization Module:

This module focuses on developing model to minimize computational time and optimize matrix solutions. By integrating image processing techniques, the system will generate matrix patterns and then apply upscaling/downscaling processes, ensuring that the structural patterns are preserved across different matrix sizes. This helps in testing the scalability and effectiveness of the solution on matrices of varying dimensions.

System Design:

The system employs a variety of methods to enhance matrix processing. Deep learning models and other techniques will be used to generate matrices and predict the optimal properties to maximize computational efficiency without sacrificing accuracy.

Furthermore, matrix patterns can be manipulated with image processing methods, allowing for flexible adjustments to matrix size, similar to how images are resized, but

while retaining key mathematical properties critical to solving linear systems efficiently.
()

Tools and Technologies:

Python for deep learning model development (TensorFlow/PyTorch)

Image processing libraries (e.g., OpenCV or PIL) for matrix pattern manipulation

Parallel computation techniques (CUDA, MPI, or OpenMP)

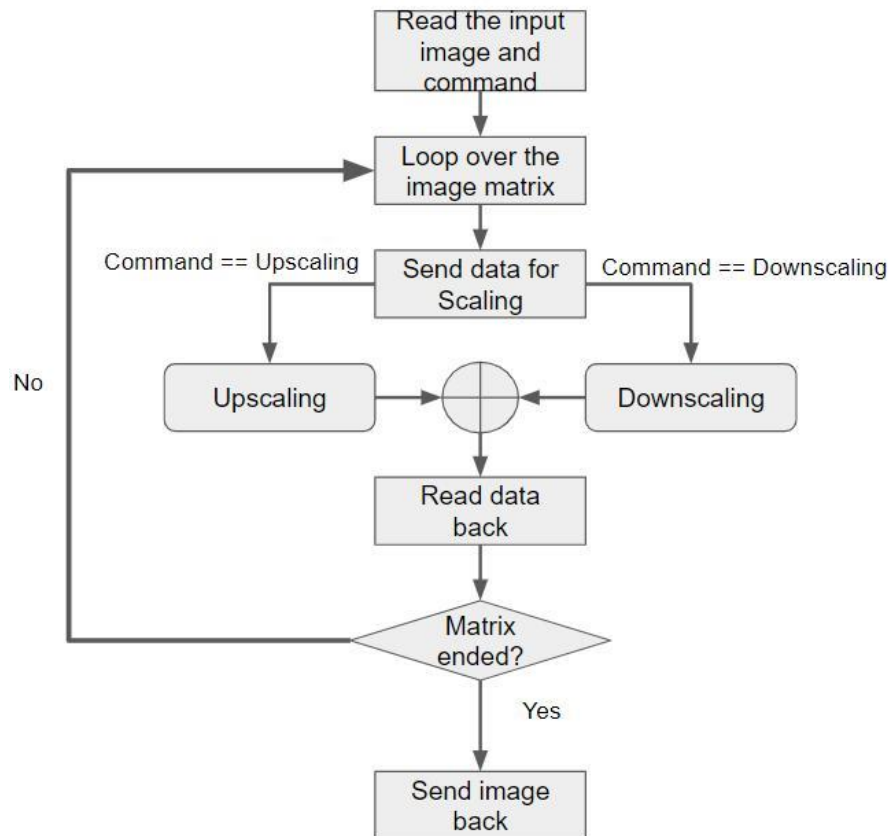
High-performance computing resources for both matrix generation and optimization

Project Timeline

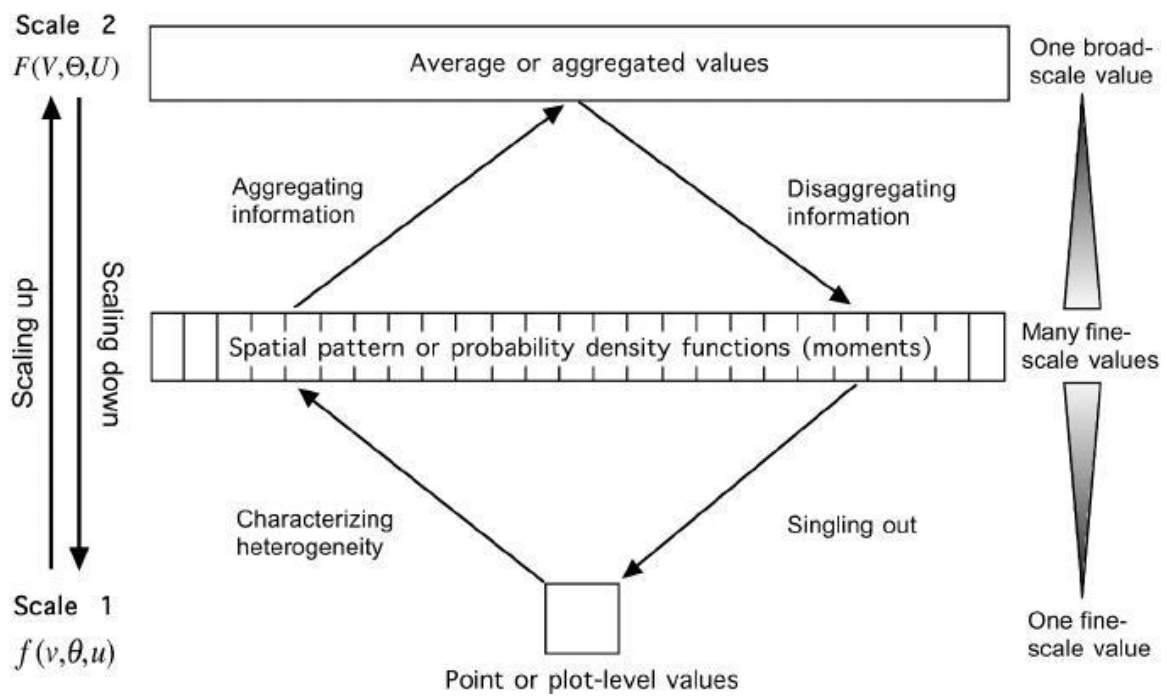
Task No	Task Name and Aim	Who Will Work on the Task	Timeframe (..-.. Months)	Success Criteria
1	Literature Review and Data Collection: Collect datasets from sparse.tamu.edu and review relevant research on sparse systems and matrix optimization.		1-2 months	Review recent papers; datasets collected.
2	Finding Different matrices		3-4 months	Find different matrices from different resources for applying the model to different fields
3	Model Development: Develop the model to generate matrices while retaining key properties.		4-6 months	Model generates matrices with low amount of error.
4	Final Report Writing and System Testing: Conduct comprehensive system testing, gathering user feedback, and compile the final project report.		7-9 months	Detailed testing report with user feedback.

4. FUTURE WORKS

Future improvements to the system could expand its applicability beyond sparse systems to other domains such as machine learning and scientific computing. Further research may focus on optimizing the model. Additionally, incorporating user feedback and conducting further testing with larger matrices under various computational conditions could lead to enhanced system performance. Future efforts could also focus on distributed computing environments and apply the method to different types of linear systems beyond those solvable by various methods.



(Fig 1.1)



(Fig 1.2)

```
function generate_sparse_matrix(m, n, sparsity, rcn):  
    # Step 1: Create a random sparse matrix A with given dimensions (m, n) and sparsity.  
    A = create_sparse_matrix(m, n, sparsity)  
  
    # Step 2: Perform Singular Value Decomposition (SVD) on the sparse matrix A.  
    U, s, Vt = sparse_svd(A, k=min(m, n) - 1)  
  
    # Step 3: Define the desired singular values range.  
    max_singular_value = 1.0  
    min_singular_value = rcn  
    singular_values = linear_space(min_singular_value, max_singular_value, length(s))  
  
    # Step 4: Reconstruct the matrix using modified singular values.  
    S = diagonal_matrix(singular_values)  
    A_mod = matrix_multiply(U, S, Vt)  
  
    # Step 5: Convert the resulting matrix back to sparse format.  
    A_sparse = convert_to_sparse(A_mod)  
  
    return A_sparse
```

(Fig 1.3)

REFERENCES

- [1] Borgatti SP, Halgin DS. On network theory.<https://doi.org/10.1287/orsc.1100.0641>
- [2] Wang JL, Chiou JM, Müller HG. Functional data analysis. Annual Review of Statistics and Its Application 2016; 3: 257–295 <https://doi.org/10.1146/annurev-statistics-041715-033624>
- [3] Wilson EL, Bathe K, Peterson F, Dovey H. Sap—a structural analysis program for linear systems. Nuclear Engineering and Design 1973; 25 (2). [https://doi.org/10.1016/0029-5493\(73\)90048-4](https://doi.org/10.1016/0029-5493(73)90048-4)
- [4] Hassan, M. A., Nar, F., & Torun, F. Ş. (2024). Leveraging machine learning to achieve efficient partitioning for a parallel hybrid solver. Conference Paper, Ankara Yildirim Beyazıt University.

[5] Torun, F. S., Manguoglu, M., & Aykanat, C. (2023). Enhancing Block Cimmino for Sparse Linear Systems with Dense Columns via Schur Complement. SIAM Journal on Scientific Computing, 45(2), C49-C72.

[6] Torun, F. Sukru, Murat Manguoglu, and Cevdet Aykanat. "A novel partitioning method for accelerating the block cimmino algorithm." SIAM Journal on Scientific Computing 40.6 (2018): C827-C850.

[7] <https://www.innovatefpga.com/cgi-bin/innovate/teams.pl?Id=AP035> (Fig 1.1)

[8] https://www.researchgate.net/figure/Upscaling-and-downscaling-as-a-two-step-process-when-a-dynamic-model-based-approach-is_fig1_260408029 (Fig 1.2)